

Support de présentation

Ce que nous disons sur chaque slide, et pourquoi nous avons fait ces choix

Projet	CryptoBot, bot de trading piloté par le machine learning
Formation	Data Engineer, DataScientest
Dépôt	DataScientest-Studio/jul26_bde_crypto
Date	3 octobre 2026

Sommaire

Déroulé

Slide 1 : CryptoBot

Slide 2 : De la donnée brute à une décision automatisée

Slide 3 : Architecture globale

Slide 4 : Collecter les données

Slide 5 : La bonne base pour la bonne forme

Slide 6 : Apprendre sans tricher

Slide 7 : Avoir raison ne suffit pas

Slide 8 : Du notebook au service protégé

Slide 9 : Ce qui fait tenir le service

Slide 10 : Démonstration en direct

Slide 11 : Tourner seul, remplacer avec preuve

Slide 12 : Du commit à la supervision

Slide 13 : Un système complet, une stratégie à améliorer

Slide 14 : Merci, questions

Pièges à éviter à l'oral

Déroulé

Deux présentateurs : Ulrich et Célian. Les passages de parole suivent les sujets que chacun maîtrise le mieux : Ulrich porte la collecte, les bases, la fiabilité du service et la chaîne de livraison ; Célian porte l'architecture, le machine learning, l'API, la démonstration, l'automatisation et la conclusion. Chaque changement de présentateur est annoncé dans la transition qui le précède. Les durées sont indicatives : la démonstration est le seul bloc à tenir à la seconde.

Slide	Sujet	Durée	Présentateur
1	Titre	0 min 30	Ulrich
2	Le projet	1 min 00	Ulrich
3	Architecture globale	1 min 30	Célian
4	Étape 1 : collecter les données	1 min 15	Ulrich
5	Étape 2 : la bonne base pour la bonne forme	1 min 30	Ulrich
6	Étape 3 : apprendre sans tricher	2 min 00	Célian
7	Le résultat, sans l'enjoliver	1 min 30	Célian
8	Étape 4 : du notebook au service protégé	1 min 30	Célian
9	Étape 4 : ce qui fait tenir le service	1 min 15	Ulrich
10	Démonstration en direct	3 min 00	Célian
11	Étape 5 : tourner seul, remplacer avec preuve	1 min 30	Célian
12	Étape 5 : du commit à la supervision	1 min 30	Ulrich
13	Bilan	1 min 30	Célian
14	Merci, questions	0 min 30	Célian
Total		20 min 00	Ulrich : 7 min 00, Célian : 13 min 00

Sept passages de parole : après les slides 2, 3, 5, 8, 9, 11 et 12. Pour qu'ils restent fluides, celui qui finit annonce le suivant par son prénom, et celui qui reprend enchaîne sans nouvelle introduction. Celui qui ne parle pas pilote l'ordinateur : il change de slide et prépare les onglets de la démonstration.

Repères de temps : la démonstration doit commencer vers 12 min 00. Si elle commence après 12 min 45, raccourcir les slides 11 et 12 à une minute chacune.

Slide 1 : CryptoBot

À aborder

Présentateur : Ulrich

- CryptoBot : projet de fin de cursus Data Engineer
- Bot de trading crypto piloté par un modèle de machine learning
- Cinq étapes : de la collecte jusqu'à l'automatisation complète
- Ulrich : collecte, bases, fiabilité, livraison ; Célian : architecture, modèle, API, démo, automatisation
- Qui porte quoi annoncé : le jury sait à qui adresser ses questions
- Fil rouge : pourquoi chaque choix, et ce qu'on a appris des erreurs
- Erreurs annoncées d'emblée : prépare la slide 7, modèle non rentable

Transition

Commençons par la question à laquelle le projet répond.

Slide 2 : De la donnée brute à une décision automatisée

À aborder

Présentateur : Ulrich

- Question : sur cinq paires Binance, la prochaine bougie monte ou baisse ?
- Bougie : ouverture, plus haut, plus bas, clôture, volume d'une tranche de temps
- Probabilité de hausse, puis décision : acheter, vendre ou attendre
- Bouton conservateur ou agressif : règle la certitude demandée avant d'agir
- Sens de la bougie plutôt que prix : R^2 trompeur de 0,999998 en recopiant le prix
- Droit d'attendre : 60 % sur peu de bougies plutôt que 52 % sur toutes
- Cinq étapes à l'écran : le parcours d'une donnée, de Binance à la décision

Si on vous relance

- Cinq paires parmi les plus échangées : bougies régulières, sans trous dus aux paires peu liquides

Transition

Avant de détailler chaque étape, Célian va vous montrer comment tout s'assemble.

Slide 3 : Architecture globale

À aborder

Présentateur : Célian

- Trois bandes : livraison du code (GitHub à la mise en service), application, supervision
- Milieu, Docker Compose : Binance, deux bases, Airflow, MLflow, API, interface
- Trois raisons de changer, code, données, modèle : une automatisation chacune, pas un script unique
- Airflow et MLflow : données qui arrivent, modèle qui vieillit ; la supervision observe tout
- API au centre, jamais exposée : tout passe par l'interface
- Une machine, seize services, une commande : démarre pareil partout, fini « ça marche chez moi »

Si on vous relance

- Profils Compose : le produit démarre toujours ; pipeline, supervision et outils à la demande

Transition

Remontons au début de la chaîne, d'où viennent les données : je laisse la parole à Ulrich.

Slide 4 : Collecter les données

À aborder

Présentateur : Ulrich

- REST : ce qui s'est passé, l'historique ; WebSocket : ce qui se passe, le temps réel
- 1 000 bougies max par appel : pagination sur la dernière reçue, juste si lot incomplet
- Quota consommé lu dans chaque réponse : jamais bannis
- WebSocket : 500 messages pour 20 bougies en 200 secondes
- Bougie clôturée seule gardée : sinon 96 % de doublons à des prix jamais confirmés
- Deux flux, un schéma commun, en UTC
- Résultat : un peu plus de deux millions de bougies, complètes à 100 %, sans doublon

Si on vous relance

- Historique commun depuis septembre 2020 : SOLUSDT n'existe que depuis août 2020
- Trous non comblés : le marché était fermé, interpoler inventerait des prix

Transition

Ces bougies, il faut maintenant les ranger, et nous avons choisi deux bases.

Slide 5 : La bonne base pour la bonne forme

À aborder

Présentateur : Ulrich

- Dimensionner d'abord : deux millions de bougies, 132 mégaoctets, très loin des limites de PostgreSQL
- Critère retenu : la forme de la donnée, pas la performance brute
- Bougie : treize colonnes fixes, clé paire, pas de temps, heure d'ouverture
- PostgreSQL et TimescaleDB : reste du SQL, tranches de temps, compression de l'ancien
- Réponses brutes de Binance, structures variables et imbriquées : MongoDB, telles qu'elles arrivent
- Couche brute : rejouer un nettoyage sans redemander six ans ; chaque bougie référence son origine
- Chargement idempotent, mise à jour sur la clé : relance sans doublon, condition de l'automatisation

Si on vous relance

- Pas « SQL pour l'historique, NoSQL pour le temps réel » : bougie REST et WebSocket, même objet
- Écartés : InfluxDB, ClickHouse (milliards de lignes, sans transactions) ; Snowflake, BigQuery (payants, non conteneurisables)
- Une table et une vue par profil : pas de doublon, le code ignore le rangement

Transition

Avec des données propres et traçables, nous pouvons entraîner un modèle sans tricher avec le temps : Célian va vous expliquer comment.

Slide 6 : Apprendre sans tricher

À aborder

Présentateur : Célian

- Fuite du futur, nous l'avons faite : contexte 1 h et 4 h, 0,52 à 0,61
- Cause : bougie de 12 h 15 nourrie par une 4 h close à 16 h
- Corrigé, retour à 0,52 ; un test automatique empêche la récursive

- Découpage chronologique, passé pour apprendre ; mesure finale sur 14 831 bougies jamais regardées
- 35 variables sans unité : un modèle appris sur le bitcoin reste valable sur l'ETH
- Forêt aléatoire calibrée : meilleur score en validation, probabilités fidèles aux fréquences observées
- Deux seuils par style : probabilité jamais au-dessus de 0,564, un seuil unique bloquait le conservateur
- Métriques entraînement contre test : accuracy 68,4 % contre 52,4 %, AUC 0,745 contre 0,535
- Écart : la forêt mémorise du bruit ; on ne juge que hors entraînement
- Hors entraînement, stable sur trois périodes : 52 à 53 %, AUC 0,535
- Quand il se prononce : 91,8 % en entraînement, 59,9 % en test

Si on vous relance

- Variables à la main, 26 puis 35 : pas les 86 colonnes de ta, 231 paires quasi identiques
- Calibration : +1,5 à +1,7 point ; gradient boosting et régression logistique moins bons en validation
- Préparation : extrait figé, variables par paire et par pas de temps, médiane pour les manques, normalisation
- R^2 , RMSE, MAPE : métriques de régression ; le naïf « prix suivant = prix actuel » fait R^2 0,99999, MAPE 0,19 %

Transition

Voici maintenant ce que vaut ce modèle, sans l'enjoliver.

Slide 7 : Avoir raison ne suffit pas

À aborder

Présentateur : Célian

- Objectif 60 % de bonnes réponses quand il se prononce : atteint, 59,9 % en conservateur
- Intervalle de confiance 55 à 64 % sur 469 ordres : nettement au-dessus du hasard
- Mais backtest à moins 3,6 % frais compris ; sans frais, environ +1 %
- Deux chiffres : une bougie de 15 min bouge de 0,227 %, l'aller-retour coûte 0,2 %
- Rentable sur 15 minutes seulement avec 94 % de bonnes réponses
- Montré tel quel, sans choisir la période : un résultat trop beau aurait signalé une fuite
- Conclusion : viser seulement les mouvements assez grands pour couvrir les frais

Si on vous relance

- Frais : 0,1 % à l'achat, 0,1 % à la vente

- Rejeu d'un an en 4 h : 53,4 % de trades gagnants, 10 154 euros
- Même réussite en 15 min : 7 547 euros ; seul le mouvement face aux frais change

Transition

Ce modèle, il faut maintenant le sortir du notebook pour le servir.

Slide 8 : Du notebook au service protégé

À aborder

Présentateur : Célian

- API FastAPI : modèle chargé une fois, rechargé seul quand le fichier est remplacé
- Une clé d'API par client, interface et Airflow : l'une se révoque sans couper l'autre
- JWT écarté : fait pour des humains avec mot de passe, nos clients sont des programmes
- Clé dans le navigateur lisible par tous : nginx ajoute la clé et transmet à l'API
- API sans aucun port ouvert sur la machine
- Fermée par défaut : sans clé configurée, réponse 503, un oubli n'ouvre rien
- Listes blanches pour paires, pas de temps et styles ; débit limité par nginx

Si on vous relance

- Limite : 10 requêtes par seconde par adresse, rafales de 40 ; protège aussi le quota Binance

Transition

Une API protégée doit aussi tenir dans la durée : Ulrich va vous montrer ce qui la rend fiable.

Slide 9 : Ce qui fait tenir le service

À aborder

Présentateur : Ulrich

- Trois piliers : tests, conteneurs, dérive
- 149 tests unitaires en une dizaine de secondes, doublures : un échec pointe le code
- Un test parcourt toutes les routes : chacune refuse un appel sans clé
- 21 vérifications sur l'application démarrée : ce que les tests unitaires ignorent, la clé nginx
- Seize services, une commande ; modèle monté à part en lecture seule, sans reconstruire l'image
- Dérive PSI par paire et pas de temps : référence mélangée, fausses alertes
- Tout le marché dérive, surtout la volatilité ; dérive ne veut pas dire baisse de performance

Si on vous relance

- Rafale de 120 appels : environ 75 refusés
- Trade moyen : 0,005 sur le bitcoin, 115,8 sur le XRP ; fenêtre de 90 jours chacun

Transition

Plutôt que de le décrire, voyons le service tourner : Célian vous fait la démonstration.

Slide 10 : Démonstration en direct

À aborder

Présentateur : Célian

- Interface du bot, servie par nginx sur la machine
- Bougies Binance en direct, décision sur la dernière bougie clôturée, carnet de positions virtuelles
- Carnet : juger le modèle en euros et en trades, pas seulement en pourcentage
- Take profit et stop loss à trois fois la volatilité récente, échéance de douze bougies
- Fermeture sur le premier des trois atteint ; les deux dans une bougie : stop loss retenu
- Stop loss par défaut : les données ne disent pas lequel est touché en premier
- Rien n'est réellement acheté : simulation sur le marché spot

Si on vous relance

- Bibliothèque de graphique livrée avec le projet : la page marche sans Internet

Script de la démonstration (3 minutes)

Avant la soutenance : application démarrée depuis au moins une heure, onglets ouverts dans cet ordre : interface (localhost:8080), Grafana (localhost:3000), Airflow (localhost:8088), MLflow (localhost:5000). Interface réglée sur BTCUSDT, 1h, Conservateur, période 1 semaine, Trades coché, Données « Binance en direct ».

Temps	Ce qu'on clique	Ce qu'on dit
0:00	Onglet interface, rien à cliquer	BTCUSDT en 1 h ; dernière bougie en direct, WebSocket ; en haut : prix, décision, probabilité, prochaine clôture
0:30	Montrer le panneau de décision et sa réglette	Probabilité de hausse ; deux seuils conservateurs : acheter, vendre, attendre ; le plus souvent, il attend
0:55	Bouton Agressif	Seuils resserrés : 0,5346 achat, 0,4315 vente ; même modèle, autre certitude
1:20	Période : 1 mois	Carnet sur un mois : flèche ouverture, rectangles take profit et stop loss, point fermeture frais déduits
1:55	Survoler deux ou trois points de fermeture	Gains et pertes ; bilan sous le graphique : une chance sur deux, frais défavorables
2:20	Paire : SOLUSDT, puis bouton 15m	Autre paire, autre pas de temps ; modèle commun aux trois pas de temps du day trading
2:45	Revenir à 1h, conservateur	Airflow appelle l'API toutes les 15 minutes, 30 combinaisons, même sans spectateur ; vers l'automatisation

Plan B si la démonstration plante

- Bougies Binance absentes (réseau coupé) : menu Données, choisir « Base du projet ». Le graphique se recharge depuis PostgreSQL ; dire « sans réseau, l'interface lit la base du projet ».
- Interface inaccessible : passer directement sur Grafana (localhost:3000), tableau « CryptoBot, production ». Montrer les décisions du bot, le taux de trades gagnants, le gain net moyen et le capital virtuel : on y voit le même carnet.
- Rien ne démarre : captures d'écran de l'interface et de Grafana, préparées la veille dans un dossier ouvert d'avance. Les commenter avec le même texte que le tableau ci-dessus, sans s'excuser plus d'une phrase.
- Dans tous les cas, ne pas déboguer en direct : au-delà de 20 secondes de blocage, passer au plan suivant.

Transition

Ce que vous venez de voir tourne sans nous : passons à l'automatisation.

Slide 11 : Tourner seul, remplacer avec preuve

À aborder

Présentateur : Célian

- Étape 5 : l'application tourne sans aucune commande lancée à la main
- Airflow : collecte et bot toutes les 15 min, dérive chaque matin, réentraînement le dimanche
- Airflow ne calcule rien : il lance les commandes du projet, testées ailleurs
- Réentraîné ne veut pas dire meilleur : remplacer à l'aveugle dégraderait la production

- Duel champion contre challenger : mêmes 21 derniers jours, jamais vus par le challenger
- Remplacement seulement si au moins aussi bon, sur au moins 30 ordres
- Deux duels, l'ancien garde sa place : écarts dans la marge d'erreur

Si on vous relance

- Extrait figé, signé SHA-256 : trois relances, mêmes chiffres à la quatrième décimale
- Publication par renommage : l'API ne lit jamais un fichier à moitié écrit, l'ancien est archivé
- Exécuteur local plutôt que Celery : quelques tâches par quart d'heure, sans file de messages

Transition

Reste à faire évoluer le code lui-même, et à savoir quand quelque chose casse : c'est la partie d'Ulrich.

Slide 12 : Du commit à la supervision

À aborder

Présentateur : Ulrich

- Push sur la branche principale : GitHub Actions, cinq contrôles en parallèle
- Analyse du code, 149 tests, DAG Airflow, configurations, démarrage complet sur machine vierge
- Contrôle des DAG : un DAG cassé disparaît sans erreur, la collecte s'arrête en silence
- Tout vert : images publiées, Watchtower les récupère ; les bases ne sont jamais remplacées
- Mode pull : la machine va chercher, rien d'ouvert ; dépôt public, pas d'exécuteur local
- Prometheus, plus un exportateur qui traduit l'état du pipeline, stocké en base, en métriques
- Douze alertes, quatre questions : service, données, bot, modèle ; puis le tableau Grafana

Si on vous relance

- Démarrage en CI avec des secrets neufs : le projet démarre ailleurs que chez nous
- Exportateur séparé : les questions importantes sont en base, il survit à une panne de l'API
- Alertes avec durée minimale, de 2 minutes à 1 heure ; Grafana en lecture seule

Partage d'écran Grafana (30 secondes)

Onglet Grafana, tableau « CryptoBot, production ». Montrer de haut en bas : l'état des services et les alertes en cours, le retard des données par série, l'évolution de la dérive, puis l'historique des réentraînements. Phrase à dire : « Tout ce que nous avons présenté se lit ici, au même endroit, et le tableau est décrit dans un fichier versionné : il est identique sur toute machine. »

Transition

Pour finir, Célian va vous dire ce que nous retenons de ce projet.

Slide 13 : Un système complet, une stratégie à améliorer

À aborder

Présentateur : Célian

- Ce qui fonctionne : chaîne complète autonome, API protégée, code en production sans intervention
- Leçon 1, valider dans le temps : nos trois plus beaux résultats étaient faux
- Leçon 2, mesurer après les coûts : raison six fois sur dix, et pourtant perdant
- Leçon 3, superviser la donnée : alertes les plus utiles, données qui arrivent, bot qui tourne
- Piste 1, changer de cible : ignorer les petits mouvements, mangés par les frais
- Piste 2, notifications Alertmanager : les 12 alertes se consultent, personne n'est prévenu
- Piste 3, sortir d'une seule machine, point faible : si elle s'arrête, tout s'arrête

Si on vous relance

- Cible plutôt que données : calendrier, carnet d'ordres, horizons plus longs n'ont rien apporté de régulier
- Avant toute ouverture au réseau : serveur avec sauvegarde des volumes, HTTPS devant nginx

Transition

Il me reste à vous remercier.

Slide 14 : Merci, questions

À aborder

Présentateur : Célian

- Merci ; une chaîne données et machine learning complète, automatisée, supervisée
- Mesurée honnêtement, jusqu'à montrer que la stratégie n'est pas encore rentable
- Quatre mots en bas de la slide : données, machine learning, API, MLOps
- Collecte, bases, tests, CI, supervision : Ulrich répond
- Architecture, modèle, API, automatisation : Célian répond ; évite de se couper la parole
- Application toujours en marche : interface, Grafana, Airflow ou MLflow rouvrables pour illustrer une question

Transition

Laisser la première question venir, sans relancer.

Pièges à éviter à l'oral

- Ne jamais dire que le bot est rentable. Dire « 60 % de bonnes réponses quand il se prononce, mais moins 3,6 % au backtest frais compris ».
- Ne pas confondre dérive des données et baisse de performance. Toutes les séries dérivent, et le modèle en service gardait 57,5 % de bonnes réponses, dans la marge des 60 %. La dérive déclenche une comparaison, jamais une mise en service automatique.
- Dire « marché spot », et préciser que le projet est académique : aucun ordre réel n'est passé, les positions sont virtuelles. Ne pas parler de contrats à terme ni d'effet de levier.
- Ne pas présenter 60 % comme une certitude : c'est une moyenne, avec un intervalle de confiance de 55 à 64 %.
- Ne pas dire que l'API est « sécurisée » sans nuance : elle n'a pas de HTTPS, ce qui est acceptable tant que tout reste sur la machine locale, et les secrets sont dans un fichier `.env`.
- Ne pas dire qu'Airflow entraîne le modèle : il lance les commandes du projet, la logique est dans les scripts testés.
- Ne pas lire les slides. Chaque chiffre affiché doit venir avec sa raison : pourquoi ce choix, et ce que nous avons écarté.
- Pendant la démonstration, ne pas déboguer en direct : passer au plan B au bout de 20 secondes.